ARQUITECTURA DE SISTEMAS CONVERSACIONALES

Diseñar antes de programar.

PROBLEMA → REQUISITOS → RESPONSABILIDADES → ARQUITECTURA → ESTRUCTURA → CÓDIGO

Caso conductor: Hotel El Ronquido Feliz

El hotel quiere desarrollar un asistente conversacional capaz de:

- ✓ responder preguntas
- ✓ realizar reservas
- ✓ comunicarse mediante WhatsApp
- ✓ consultar disponibilidad
- ✓ cancelar reservas
- ✓ integrarse con sistemas externos

UN MODELO NO ES UNA APLICACIÓN

Un sistema sencillo puede parecer:



Pero una aplicación real puede necesitar además:

- información actualizada
- acceso a datos
- acciones reales
- servicios externos
- estado
- reglas de negocio
- seguridad
- gestión de errores

El LLM es un componente del sistema. No es el sistema completo.

Por ejemplo: “¿A qué hora abre la piscina?” puede resolverse con conocimiento/contexto. Pero “Reserva una habitación para el viernes” requiere consultar información real y ejecutar una acción real. Una frase generada por el LLM no crea por sí sola una reserva.

CUANDO APARECE EL SISTEMA

Para convertir un modelo en una aplicación necesitamos responder:

1

¿Qué piezas necesitamos?

2

¿Qué responsabilidad tiene cada pieza?

3

¿Cómo se comunican?

4

¿De qué dependen?

Definiciones

Componente: parte diferenciada del sistema.

Responsabilidad: trabajo que corresponde a un componente y límites de lo que debe hacer.

Relación: forma en la que dos componentes colaboran.

Dependencia: elemento que un componente necesita para funcionar.

¿QUÉ ES ARQUITECTURA DE SOFTWARE?

Arquitectura de software

Diseño de alto nivel que define:

- los principales componentes del sistema;
- sus responsabilidades;
- sus relaciones;
- sus dependencias;
- las decisiones estructurales relevantes.

La arquitectura responde preguntas como:

- ¿Dónde vive cada responsabilidad?
- ¿Qué partes deben permanecer separadas?
- ¿Qué depende de qué?
- ¿Cómo circula la información?
- ¿Qué ocurre cuando algo cambia o falla?

Arquitectura ≠ escribir código.

ARQUITECTURA NO ES TECNOLOGÍA

No deberíamos empezar preguntando:

¿LangChain o LangGraph? · ¿OpenAI o Anthropic? · ¿PostgreSQL o MongoDB? · ¿AWS o Azure?

Primero:

¿Qué problema necesitamos resolver?

Después:

¿Qué necesita hacer nuestro sistema?

Finalmente:

¿Qué tecnología nos ayuda a implementarlo?

ANTES DE DISEÑAR: REQUISITOS

Antes de diseñar preguntamos:

¿Qué debe hacer el sistema?

→ **Requisitos funcionales**

¿Cómo debe hacerlo o bajo qué restricciones?

→ **Requisitos no funcionales**

REQUISITOS FUNCIONALES

Requisito funcional

Describe una capacidad o comportamiento que el sistema debe proporcionar.

Pregunta que responde:

¿QUÉ debe hacer el sistema?

Para el hotel:

- **RF1.** Responder preguntas sobre el hotel.
- **RF2.** Consultar disponibilidad.
- **RF3.** Crear una reserva.
- **RF4.** Cancelar una reserva.
- **RF5.** Enviar confirmaciones al cliente.
- **RF6.** Mantener una conversación con el usuario.

REQUISITOS NO FUNCIONALES

Requisito no funcional

Describe condiciones, restricciones o propiedades de calidad del sistema.

Pregunta que responde:

¿CÓMO de bien o BAJO QUÉ CONDICIONES debe funcionar?

Ejemplo:

Una caída de WhatsApp no debería provocar la pérdida de una reserva ya creada.

Si exijo mucha disponibilidad, seguridad o baja latencia, eso puede hacer que tenga que diseñar el sistema de otra forma.

Ejemplos:

- seguridad
- disponibilidad
- escalabilidad
- trazabilidad
- rendimiento
- mantenibilidad
- coste

EL PASO QUE NO DEBEMOS SALTARNOS

REQUISITO → RESPONSABILIDAD

Un requisito puede generar varias responsabilidades.

Requisito:

“El usuario debe poder reservar una habitación.”

Pregunta que hacemos:

¿Qué cosas tienen que pasar para que esta funcionalidad ocurra de principio a fin?

Descomponer en pasos nos ayudará a identificar responsabilidades:

- recibir la petición
- entender qué quiere el usuario
- recopilar los datos necesarios
- consultar disponibilidad
- validar las reglas de reserva
- crear la reserva
- guardar la información
- comunicar el resultado

PASO ≠ RESPONSABILIDAD

Un paso describe algo que ocurre en el proceso.
Una responsabilidad describe un trabajo del que alguna parte del sistema debe hacerse cargo.
No hace falta identificar todavía todas las responsabilidades perfectas.
Primero descomponemos el requisito en pasos. Más adelante, al agruparlos en componentes y capas, veremos con más claridad qué responsabilidades existen realmente.

RESPONSABILIDAD, ACOPLAMIENTO Y COHESIÓN

Una separación suele ser útil cuando dos partes:

- ✓ hacen trabajos diferentes;
- ✓ cambian por motivos diferentes;
- ✓ dependen de tecnologías diferentes;
- ✓ necesitan probarse o evolucionar de forma independiente.

Pregunta útil:

"Si esto cambia, ¿qué otras cosas tendría que tocar?"

Bajo acoplamiento:

Una parte conoce solo lo necesario de otra.

Alta cohesión:

Las cosas que pertenecen a una misma responsabilidad permanecen juntas.

Ejemplo:

La regla "una habitación no puede reservarse dos veces" no debería depender de WhatsApp, OpenAI o PostgreSQL.

EL SISTEMA DESDE ARRIBA



Importante:

Este esquema no representa una única arquitectura posible. Es un mapa conceptual de responsabilidades que nos ayuda a entender el sistema antes de implementarlo.

¿DÓNDE ESTÁ LA IA?

El LLM puede encargarse de tareas como:

- interpretar lenguaje
- clasificar intenciones
- extraer información
- generar respuestas
- ayudar a seleccionar una acción

Pero no debería utilizarse como fuente de verdad para operaciones críticas.

Ejemplo:

LLM: “Parece que quieres reservar para el viernes.”

Sistema de reservas: “Existe disponibilidad real para el viernes.”

Lenguaje y realidad operativa no son lo mismo.

LOS PATRONES APARECEN PORQUE APARECEN PROBLEMAS

Patrón de diseño

Solución conocida y reutilizable para un tipo de problema de diseño que aparece de forma recurrente.

No es:

- una librería · un framework · código que debamos copiar literalmente

Pregunta clave:

¿Para qué sirven? → Para no reinventar una solución desde cero cada vez que aparece un problema conocido.

Importante:

No buscamos memorizar todos los patrones. Buscamos reconocer el problema y saber que existen soluciones conocidas que podemos estudiar y aplicar cuando hagan falta.

¿POR QUÉ VEMOS ESTOS PATRONES?

Existen muchos patrones. Hoy solo veremos algunos ejemplos.

¿Por qué estos?

- **Strategy:** ayuda a pensar en comportamientos/implementaciones intercambiables.
- **Adapter:** ayuda a integrar sistemas con interfaces diferentes.
- **Repository:** ayuda a separar acceso a datos de la lógica del negocio.

Y en la parte agentic veremos ReAct con más detalle porque:

1. forma parte explícita de esta unidad;
2. ayuda a entender muy bien el ciclo de un agente que decide, usa una herramienta, observa el resultado y vuelve a decidir.

No significa que ReAct sea “el mejor patrón”. Es el patrón en el que profundizaremos hoy.

STRATEGY: VARIAS FORMAS DE HACER EL MISMO TRABAJO

Problema:

Tengo varias formas intercambiables de realizar un comportamiento y quiero poder cambiar entre ellas sin reescribir quien las utiliza.

Pregunta para reconocerlo:

“¿Quiero realizar el mismo tipo de trabajo de distintas maneras intercambiables?”

Ejemplo cotidiano:

Madrid → Barcelona
Coche / Tren / Avión

Ejemplo software:

USAR UN MODELO
→ proveedor A / proveedor B / proveedor C

ADAPTER: HACER QUE DOS PIEZAS ENCAJEN

Problema:

Dos piezas necesitan colaborar, pero sus interfaces no encajan.

Pregunta para reconocerlo:

“¿Tengo algo que sabe hacer lo que necesito, pero “habla” una interfaz diferente?”

Ejemplo cotidiano:

Enchufe europeo → ADAPTADOR → toma americana

Ejemplo software:

Nuestra aplicación: enviar_mensaje(texto)

↓

WhatsAppAdapter

↓

API de WhatsApp con su propio formato

REPOSITORY: SEPARAR DATOS Y NEGOCIO

Problema:

No quiero que las reglas del negocio estén llenas de detalles sobre cómo se almacenan o recuperan datos.

Repository ofrece una abstracción para acceder a esos datos.

Pregunta para reconocerlo:

"¿Mi lógica necesita datos pero no debería conocer SQL, el motor de base de datos o el detalle de persistencia?"

Ejemplos de operaciones:

- eliminar_reserva(id)
- guardar_reserva(reserva)
- buscar_reserva(id)

EN SISTEMAS AGENTIC TAMBIÉN HAY PATRONES

Además de patrones generales de software, existen patrones para organizar flujos con LLMs/agentes.

Ejemplos:

- **Prompt chaining / Pipeline** → pasos conocidos en secuencia
- **Routing** → elegir entre varios caminos
- **Planner-Executor** → planificar y después ejecutar
- **Evaluator-Optimizer / Reflection** → generar, evaluar y mejorar
- **Orchestrator-Workers** → repartir trabajo entre especialistas
- **Agent loop / ReAct** → decidir, actuar, observar y volver a decidir

No son opciones excluyentes: un sistema puede combinar varios.

Moraleja:

No hay que memorizar todos. Hay que saber qué problema resuelve cada uno y consultar cómo implementarlo cuando sea necesario.

ABRIMOS LA CAJA CONVERSACIONAL

Agente: sistema en el que el modelo puede usar contexto/estado y herramientas y decidir acciones para avanzar hacia un objetivo.

Un sistema conversacional/agentic puede necesitar:

LLM

Interpreta y genera lenguaje.

Instrucciones / contexto

Información relevante para realizar la tarea.

Estado

Información que debe mantenerse entre pasos o turnos.

Herramientas (tools)

Capacidades para consultar o actuar sobre sistemas reales.

Orquestación

Coordina el flujo: qué hacer y qué ocurre a continuación.

EL AGENTE VIVE DENTRO DE LA ARQUITECTURA

Arquitectura general del sistema

→ organiza responsabilidades del sistema completo.

Dentro puede existir un componente agentic.

En nuestro ejemplo, ese componente puede incluir conceptualmente: orquestación, modelo de lenguaje, estado/contexto y herramientas.

Y el agente puede seguir un patrón de comportamiento como ReAct.

Idea clave:

El agente no sustituye a la arquitectura. El agente vive dentro de la arquitectura.

CUANDO RESPONDER NO ES SUFICIENTE

Usuario:

“Quiero una habitación para el viernes para dos personas.”

El sistema no debería responder inmediatamente:

“Reserva realizada.”

Primero necesita actuar sobre información real.



¿Por qué ReAct encaja aquí?

ReAct puede encajar cuando el modelo necesita decidir dinámicamente qué acción realizar a partir de lo que va observando.

REACT (Reasoning + Acting) PASO A PASO: DECIDIR

Situación:

Disponemos de:

- fecha = viernes
- personas = 2

Nos falta:

- disponibilidad = ?

Pregunta:

¿Qué necesitamos saber antes de continuar?

Respuesta:

La disponibilidad real.

Decisión:

Utilizar la herramienta:
consultar_disponibilidad(fecha, personas)

REACT PASO A PASO: ACTUAR Y OBSERVAR

Acción:

consultar_disponibilidad(viernes, 2)

Observación:

Habitación disponible.
Precio: 120 €.

Pregunta:

¿Qué ha cambiado?

Respuesta:

Ahora el sistema posee información real que antes no tenía. Esa observación puede cambiar la siguiente decisión.

REACT PASO A PASO: VOLVER A DECIDIR

Ciclo:

DECIDIR
↓
ACTUAR
↓
OBSERVAR
↓
DECIDIR
↓
...

Ejemplo:

Observación: hay habitación por 120 €.

Nueva decisión: pedir confirmación al usuario.
Después de confirmar: crear_reserva().

ReAct:

Patrón que intercala decisión/planificación y acciones, utilizando las observaciones obtenidas para decidir los siguientes pasos.

Importante: ReAct no es una arquitectura completa. Es una forma de organizar parte del comportamiento del sistema.

DE LA IDEA AL CÓDIGO: SOLO PARA VERLO

No hay que memorizar este código. Solo muestra cómo la idea puede traducirse a una estructura de control.

```
PARA paso DESDE 1 HASTA MAX_PASOS:

   accion = decidir_con_llm(estado)         # DECIDIR

   SI accion es RESPONDER:
      enviar accion.mensaje
      TERMINAR

   resultado = ejecutar_herramienta(accion)  # ACTUAR

   estado = actualizar_estado(
      estado,
      accion,
      resultado
   )                                 # OBSERVAR


SI se agotaron los pasos:
   responder con un fallback o derivar a una persona
```

El LLM decide la siguiente acción, una herramienta la ejecuta, el sistema observa el resultado y actualiza el estado.

Concepto → diseño → implementación

La implementación real dependerá del framework, lenguaje y restricciones del proyecto.

AHORA SÍ: UNA REFERENCIA DE CAPAS

Hay muchas formas de estructurar una aplicación. Para esta clase utilizaremos como referencia un modelo de cuatro capas documentado por Microsoft:

PRESENTATION
Interacción/entrada-salida con usuarios o sistemas externos.

APPLICATION
Coordina casos de uso y flujo de aplicación.

DOMAIN
Conceptos y reglas propias del negocio.

INFRASTRUCTURE
Detalles técnicos: persistencia, proveedores, APIs e integraciones.

Importante: No es la única arquitectura correcta. La usamos porque permite aprender con claridad la separación de responsabilidades.

¿DÓNDE ENCAJA CADA RESPONSABILIDAD?

Checklist didáctica

¿Interacción con el exterior / entrada-salida?
→ PRESENTATION

¿Coordina los pasos de un caso de uso?
→ APPLICATION

¿Es una regla o concepto propio del hotel?
→ DOMAIN

¿Es un detalle técnico o integración externa?
→ INFRASTRUCTURE

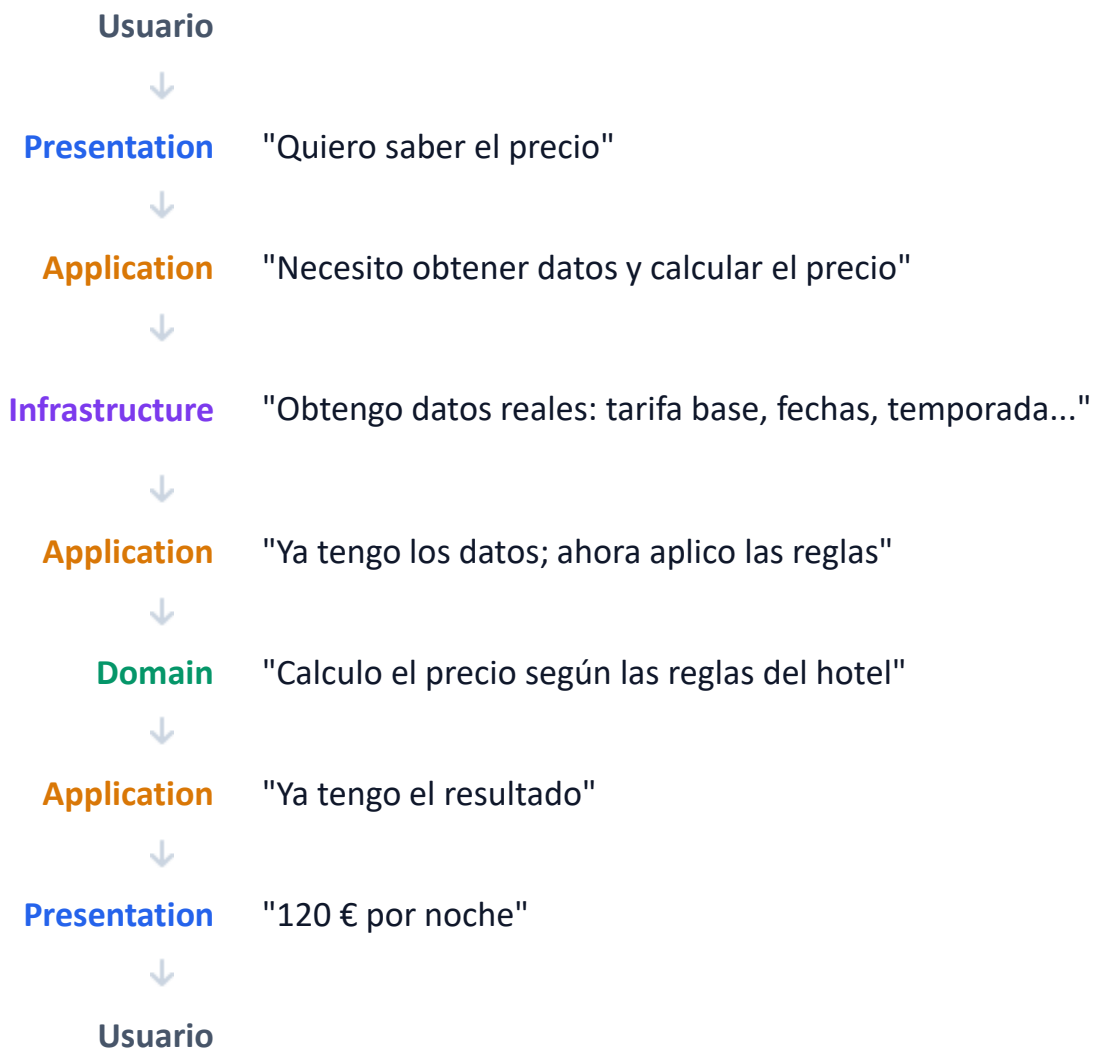
Pregunta importante:

¿Y el agente?

Respuesta:

No es una “quinta capa”. En nuestro ejemplo, su orquestación encaja principalmente en Application; las integraciones concretas que utiliza encajan en Infrastructure. En otros diseños puede organizarse de otra manera.

CASO 1: CÁLCULO DE PRECIO



Presentation

¿cómo entra y sale la información?

Application

¿qué pasos hay que coordinar?

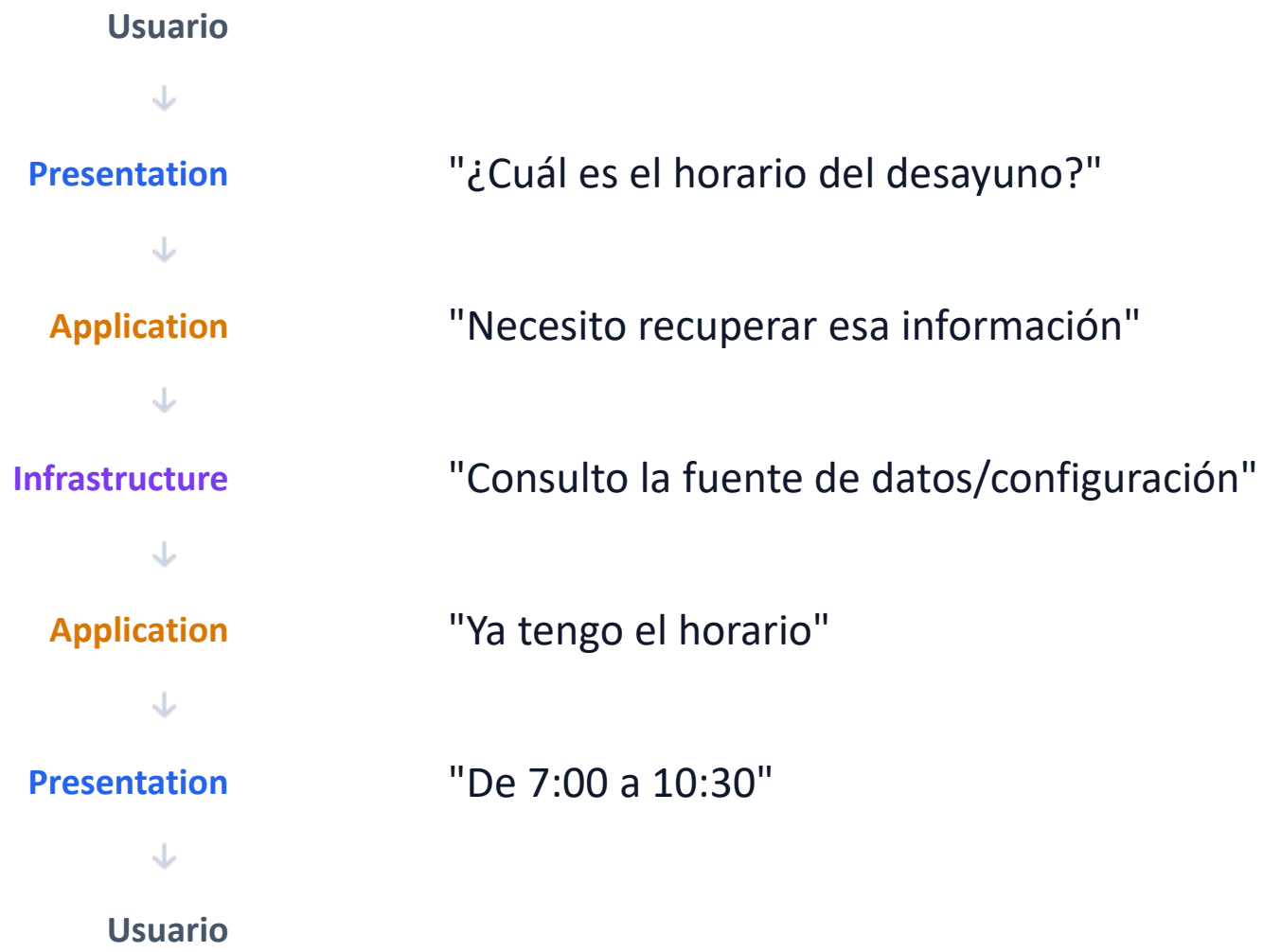
Domain

¿qué reglas propias del hotel hay que aplicar?

Infrastructure

¿con qué sistemas o tecnologías tengo que hablar?

CASO 2: CONSULTA DE INFORMACIÓN



NO EXISTE UNA ESTRUCTURA UNIVERSAL DE CARPETAS

La arquitectura lógica y la estructura física del código están relacionadas, pero no son lo mismo. Un proyecto puede organizarse, por ejemplo:

POR CAPAS
presentation/ · application/ · domain/ · infrastructure/

POR FUNCIONALIDADES / FEATURES
reservas/ · clientes/ · pagos/

POR COMPONENTES O CAPACIDADES DE IA
agents/ · tools/ · retrieval/ · workflows/

HÍBRIDA
combina varios criterios cuando el proyecto lo necesita.

Lo común debajo de todas ellas: separar responsabilidades, controlar dependencias y agrupar cosas relacionadas.

DISTINTAS FORMAS DE ORGANIZAR LAS MISMAS RESPONSABILIDADES

Comparación aproximada. **NO es una equivalencia 1:1.**

Visión por capas

Presentation
Application
Domain
Infrastructure

En una estructura orientada a capacidades

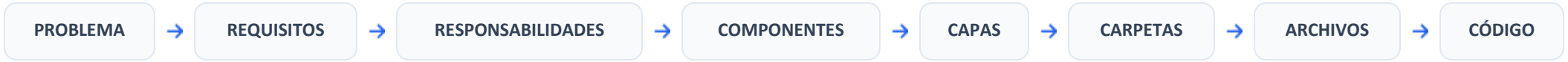
api/, web/, cli/
agents/, workflows/, orchestrators/, chains/
reglas de negocio, modelos propios, validaciones
tools/, llm_clients/, db/, vector_store/, APIs

Aunque cambien los nombres de las carpetas, seguimos pudiendo hacernos las mismas preguntas:

- ¿Quién recibe la petición?
- ¿Quién coordina?
- ¿Dónde están las reglas?
- ¿Qué pieza habla con servicios externos?
- ¿Qué responsabilidad tiene cada componente?

Las carpetas pueden cambiar. Las responsabilidades siguen existiendo

LA CADENA COMPLETA: DE PROBLEMA A CÓDIGO (Hagamos zoom…)



IDEA FUNDAMENTAL

Las carpetas no son la arquitectura. Las carpetas reflejan decisiones arquitectónicas.

CONSTRUIMOS NUESTRO PROYECTO DESDE CERO

Partimos de cuatro responsabilidades generales:

- interacción con el usuario → **Presentation**
- coordinación de casos de uso → **Application**
- reglas y conceptos del hotel → **Domain**
- integraciones y detalles técnicos → **Infrastructure**

No es una plantilla universal. Es una estructura sencilla coherente con el modelo conceptual elegido.

```
hotel-ai/
├── src/
│   ├── presentation/
│   ├── application/
│   ├── domain/
│   └── infrastructure/
├── tests/
├── docs/
└── README.md
```

LOS ARCHIVOS NO APARECEN DE LA NADA

Regla: Primero aparece una responsabilidad. Después creamos una pieza de código que se haga cargo de ella.

Recibir mensajes del chat
→ presentation/chat_routes.py

Coordinar una reserva
→ application/create_reservation.py

Representar una reserva
→ domain/reservation.py

Aplicar reglas de reserva
→ domain/reservation_rules.py

Consultar el sistema de reservas
→ infrastructure/booking_client.py

Hablar con el proveedor LLM
→ infrastructure/llm_client.py

Los nombres son ejemplos. Lo importante es justificar la responsabilidad.

BAJAMOS UN NIVEL: ESTRUCTURA RESULTANTE

```
hotel-ai/
└── src/
    ├── presentation/
    │   └── chat_routes.py
    ├── application/
    │   ├── create_reservation.py
    │   └── reservation_agent.py
    ├── domain/
    │   ├── reservation.py
    │   └── reservation_rules.py
    ├── infrastructure/
    │   ├── booking_client.py
    │   └── llm_client.py
    ├── config.py
    └── main.py
```

¿Por qué no creamos subcarpetas agents/, llm/, persistence/…?

Porque todavía tenemos muy pocas piezas. Crear carpetas por adelantado añade estructura sin necesidad.

Si el proyecto crece, la estructura puede evolucionar.

¿CUÁNDO CREAR UNA SUBCARPETA?

No crear carpetas por adelantado.

Crear una agrupación cuando:

- existen varias piezas relacionadas;
- comparten una responsabilidad o propósito claro;
- mejora la comprensión del proyecto;
- probablemente evolucionarán juntas.

Ejemplo:

Al principio:

```
infrastructure/
├── llm_client.py
└── whatsapp_client.py
```

Si crece:

```
infrastructure/
├── llm/
│   ├── openai_client.py
│   └── anthropic_client.py
└── messaging/
    ├── whatsapp_client.py
    └── email_client.py
```

La estructura evoluciona con el sistema.

CASO FINAL GUIADO: NUEVA PETICIÓN DEL HOTEL

Nueva petición:

“El cliente podrá pedir desde el chat añadir desayuno a una reserva existente.”

Vamos a resolverlo juntos, sin escribir código.

Seguiremos siempre el mismo camino:

1. Requisitos
2. Responsabilidades
3. ¿Necesitamos comportamiento agentic / ReAct?
4. Capas
5. Archivos
6. Estructura final

CASO FINAL — PASO 1: REQUISITOS

Pregunta: ¿Qué debe hacer el sistema?

Respuesta — requisitos funcionales:

- identificar la reserva indicada por el cliente;
- consultar la reserva real;
- comprobar si se puede añadir desayuno;
- añadir el desayuno;
- confirmar el resultado al usuario.

Pregunta: ¿Qué condiciones queremos respetar?

Respuesta — requisitos no funcionales / restricciones simples:

- el sistema debe responder en menos de 2 segundos en el 90% de los casos (rendimiento);
- la información del cliente debe almacenarse cifrada (seguridad);
- las modificaciones de reservas deben quedar registradas para mantener la trazabilidad (autoría/ trazabilidad).

CASO FINAL — PASO 2: RESPONSABILIDADES

Pregunta: ¿Qué cosas tienen que pasar de principio a fin?

Respuesta:

Recibir mensaje
↓
Interpretar petición
↓
Identificar reserva
↓
Consultar reserva real
↓
Aplicar regla “¿se puede añadir desayuno?”
↓
Solicitar modificación
↓
Observar resultado
↓
Responder al usuario

Idea:

Cada paso nos ayuda a identificar que responsabilidades necesita el sistema.

Responsabilidades que descubrimos:

recibir mensaje / responder → entrada-salida

interpretar petición → gestión conversacional

identificar reserva / solicitar modificación / observar resultado → coordinación

comprobar si se puede añadir desayuno → reglas de negocio

consultar reserva real / modificar sistema externo → integración con sistema externo de reservas

CASO FINAL — PASO 3: ¿REACT TIENE SENTIDO?

Pregunta:

¿El siguiente paso depende del resultado de una acción anterior?

Respuesta:

Sí. Primero debemos consultar la reserva real. Según el resultado, decidiremos qué hacer.

Ejemplo:

```
DECIDIR: necesito consultar la reserva
↓
ACTUAR: consultar_reserva(id)
↓
OBSERVAR: reserva encontrada / no encontrada
↓
DECIDIR:
 • si no existe → informar
 • si existe → comprobar regla y continuar
↓
ACTUAR: añadir_desayuno(id)
↓
OBSERVAR: modificación aceptada / rechazada
↓
RESPONDER
```

Conclusión: ReAct puede ser una forma adecuada de organizar esta parte dinámica del comportamiento.

ACLARACIÓN MUY IMPORTANTE: Si bien en el ejemplo indico que puede ser una forma adecuada, esta es una decisión arquitectónica de verdad: no metemos ReAct porque sí. Para añadir desayuno, un workflow puede ser más simple y predecible. ReAct tiene sentido cuando necesitamos flexibilidad y decisiones . Y recordad: más comportamiento agentic normalmente supone también más latencia y coste. O sea… una vez dicho esto… ¿Sigo pensando que es adecuado ReAct para el caso de uso planteado?... Quizás no! Pero lo usaré para seguir explicando la clase..

CASO FINAL — PASO 4: ¿DÓNDE VIVE CADA RESPONSABILIDAD?

¿Quién recibe el mensaje?
→ **Presentation**

¿Quién coordina consultar, comprobar y modificar?
→ **Application**

¿Dónde vive la regla “se puede añadir desayuno”?
→ **Domain**

¿Quién habla con el sistema real de reservas?
→ **Infrastructure**

¿Dónde encaja el comportamiento del agente en nuestro ejemplo?

Principalmente en Application como coordinación; las tools no son una capa arquitectónica y pueden invocar casos de uso de Application, mientras que las integraciones técnicas concretas se implementan normalmente en Infrastructure.

Importante: Ésta es nuestra organización didáctica, no una regla universal para todos los frameworks.

CASO FINAL — PASO 5: ¿QUÉ ARCHIVOS NECESITAMOS?

A partir de las responsabilidades identificamos qué piezas y archivos necesita el proyecto.

presentation/chat_routes.py
“Imaginemos que aquí vive el endpoint o entrada del chat.”

application/add_breakfast.py
“Imaginemos que aquí se organizan los pasos para añadir el desayuno.”

application/reservation_agent.py
“Imaginemos que aquí vive la lógica de coordinación del agente.”

domain/reservation_rules.py
“Imaginemos que aquí está la regla que decide si ese cambio está permitido.”

infrastructure/booking_client.py
“Imaginemos que aquí está el código que consulta/modifica el sistema externo.”

infrastructure/llm_client.py
“Imaginemos que aquí está el código que comunica con el LLM.”

CASO FINAL — PASO 6: RESULTADO

```
src/
├── presentation/
│   └── chat_routes.py
├── application/
│   ├── add_breakfast.py
│   └── reservation_agent.py
├── domain/
│   └── reservation_rules.py
└── infrastructure/
    ├── booking_client.py
    └── llm_client.py
```

Pregunta final:

¿Podrías justificar por qué cada archivo vive ahí?

Si puedes justificarlo por su responsabilidad, has entendido la estructura.

UNA ARQUITECTURA TAMBIÉN SE EXPLICA

Documentación mínima útil:

README

Explica qué hace el proyecto, cómo está organizado, cómo ejecutarlo y sus principales dependencias.

ADR — Architecture Decision Record

Registra una decisión arquitectónica importante: contexto / problema, decisión y consecuencias.

EJEMPLO README

Hotel El Ronquido Feliz

Asistente conversacional para atender consultas de clientes y gestionar reservas del hotel.

Funcionalidades principales

- Responder preguntas sobre el hotel.
- Consultar disponibilidad.
- Crear reservas.
- Cancelar reservas.
- Enviar confirmaciones al cliente.

Estructura del proyecto

```
src/
├── presentation/
├── application/
├── domain/
└── infrastructure/
```

Organización

- Presentation: entrada y salida de información con el usuario.
- Application: coordinación de los casos de uso.
- Domain: conceptos y reglas propias del hotel.
- Infrastructure: persistencia e integraciones técnicas.

Dependencias principales

El proyecto utiliza, entre otras:

- FastAPI
- OpenAI SDK
- SQLAlchemy
- python-dotenv

Las dependencias concretas se definen en:

requirements.txt

Instalación

```
pip install -r requirements.txt
```

Ejecución

```
python main.py
```

Documentación

Las decisiones arquitectónicas importantes se registran en:

docs/adr/

EJEMPLO ADR

ADR-001: Aislar el proveedor LLM

Estado

Aceptado

Contexto

Actualmente utilizamos OpenAI, pero en el futuro podríamos necesitar cambiar de proveedor.

No queremos que la lógica principal del hotel dependa directamente de OpenAI.

Decisión

Aislaremos la comunicación con el proveedor LLM detrás de una abstracción estable.

Las implementaciones concretas de cada proveedor se situarán en Infrastructure.

Consecuencias

Positivas:
- menor acoplamiento al proveedor;
- cambiar de proveedor tendrá menor impacto;
- facilita incorporar otros proveedores.

Negativas:
- añadimos una abstracción;
- aumenta ligeramente el número de piezas.

DE PROBLEMA A ARQUITECTURA: CHECKLIST

¿Qué quiere conseguir el usuario?

¿Qué debe hacer el sistema? → requisitos

¿Qué responsabilidades aparecen?

¿Qué pertenece al negocio y qué depende del exterior?

¿Qué necesita la parte de IA? → LLM, contexto, estado, herramientas, coordinación

¿Existe un patrón conocido que encaje con algún problema?

¿Cómo agrupamos las responsabilidades?

¿Cómo lo reflejamos en carpetas y archivos?

Moraleja:

La arquitectura no consiste en saber de memoria todas las soluciones. Consiste en analizar el problema, reconocer responsabilidades y apoyarse en soluciones conocidas cuando tienen sentido.

CINCO IDEAS PARA LLEVARSE

1. Un LLM es un componente del sistema, no el sistema completo.

2. La arquitectura organiza responsabilidades, relaciones y dependencias.

3. Los patrones son soluciones conocidas para problemas recurrentes; no hay que memorizarlos todos.

4. ReAct organiza un ciclo de decidir → actuar → observar → volver a decidir cuando el modelo necesita decidir dinámicamente los siguientes pasos; si el flujo está prefijado, puede bastar un workflow.

5. Las carpetas son consecuencia del diseño, no el punto de partida.

PROBLEMA → REQUISITOS → RESPONSABILIDADES → ARQUITECTURA → ESTRUCTURA → CÓDIGO